Creating a Paula

Digital Stables field toolkit: from a bare SD card to a working Paula with PaulaDeployer, the Nearby Devices map, and HTTPS.
Version 3 · 3 November 2026 · PaulaUploader repository, `docs/Creating_a_Paula.pdf`

Each step is marked with where it is done: **PI** on the Paula Pi (its own keyboard and screen, or SSH where it says so), **COMPUTER** on Ari's computer, **PHONE** on the phone used in the field.

1. What a Paula is
2. What you need
3. Prepare the Raspberry Pi (SD card, raspi-config)
4. Run the provisioning script
5. Trust the Pi on the NUC
6. Flash Paula's ESP32 (Wally) with Paula.ino
7. HTTPS: certificate and phones
8. Field networking: phone hotspot and Factory tunnel
9. Paula's secret and clock
10. Final checks
11. Using the Nearby Devices map
12. Updating a Paula later
13. Troubleshooting
14. Reference: files, addresses, commands

1. What a Paula is

Paula is the field toolkit taken around the farm and to customers. It has two parts that work together:

- **The Raspberry Pi** (hostname e.g. `paula3`). It runs:
  - **PaulaDeployer**, the phone web app: deploy queue, Inspect, Send Command, Nearby Devices map, Lora Sniffer, Stream 2 Chat, sensor tests. It runs in Tomcat 8.5.100 as the `pauladeployer-tomcat` service.
  - **PaulaUploader**, the command-line tool that pulls deploy packages and firmware from the office NUC and flashes devices.
  - A local Postgres database shared by both.
- **Wally**, Paula's own ESP32 board (Daffodil-style board with CP2104 USB, 433 MHz LoRa, OLED, WS2812 LEDs, PCF8563 clock), running `Paula.ino`. It listens to every LoRa packet on the farm, sends WiFi on/off commands to Daffodils, and does the bench sensor tests. It is plugged into the Pi by USB. PaulaDeployer finds it by its CP2104 USB ID (10C4:EA60) and by asking it `GetSwitchState`.

The device being flashed or inspected (a Daffodil, for example) is plugged into the Pi by a second USB cable.

> The Paula in use today is build 1 (`Paula.ino`) and has **no GPS**. On the map, the phone's GPS stands in for Paula's position. `Paula2.ino` is the build 2 board (GPS, RS485) and is not covered here except where noted.

Networks on a Paula

```
          wlan0 (built-in radio)                    wlan1 (USB WiFi dongle)
   Paula's own hotspot, SSID = hostname        joins a saved network:
   address 192.168.50.1, DHCP .10-.100           - home/factory WiFi (priority 10)
   phones/laptops join it in the field           - the phone's 4G hotspot (priority 1)
                                              (Factory tunnel only away from home)
```

PaulaDeployer answers on `http://` port 80 (redirected to 8080) and, once section 7 is done, on `https://` port 443 (redirected to 8443).

2. What you need

- Raspberry Pi (a Pi 3B or newer with built-in WiFi), a good SD card (16 GB or more) and a power supply.
- A USB WiFi dongle, which becomes `wlan1`. Without it there is no client network in the field: no tunnel, no NTP time, no map tiles through Paula.
- The Wally board with 433 MHz LoRa antenna, plus its USB cable to the Pi.
- A keyboard and screen for the Pi, for sections 3–5.
- Ari's computer, with:
  - `arduino-cli` and the ESP32 core `esp32:esp32`;
  - the libraries folder `ProjectsSupport/Esp32ArduinoLib/libraries`;
  - `openssl`;
  - the Digital Stables root certificate in `~/DigitalStablesCA` (made the first time in section 7).
- The office WiFi and the NUC (192.168.1.138) reachable during setup.
- For a customer's Paula: one of the customer's own Daffodils, to copy its secret (section 9).

3. Prepare the Raspberry Pi

COMPUTER 3.1 Flash the SD card

Use Raspberry Pi Imager to write **Raspberry Pi OS Trixie (64-bit)**. Lite is fine, since no desktop is needed. Enable SSH and set a username/password (use `pi`: the scripts and paths assume `/home/pi`). Leave the hostname and WiFi blank. They are set in the next step, which is more reliable than the Imager's fields.

PI 3.2 First boot: `sudo raspi-config`, in this order

1. **Localisation Options → WLAN Country** (Australia). Do this first: the radio does not transmit at all until a country is set.
2. **System Options → Hostname**, e.g. `paula4`. It becomes the hotspot name, the `.local` address (`paula4.local`) and the name in its HTTPS certificate. Choose it carefully. If you change it later, make a new certificate (section 7).
3. **System Options → Wireless LAN**: join the office/factory WiFi, which gives internet during setup. The provisioning script also detects it and saves it for the USB dongle.
4. Finish, reboot if asked, and check that `ip a show wlan0` shows an address.

4. PI Run the provisioning script

Run this on the Pi's own keyboard and screen, **not over SSH**. The script reconfigures the network and reboots at the end, so an SSH session drops partway through.

```
sudo apt-get update && sudo apt-get install -y git
git clone https://github.com/arifainchtein/PaulaUploader.git
cd PaulaUploader
chmod +x provision-pi.sh
./provision-pi.sh
```

The script:

- prints the commit it runs from first; check that it matches the latest on GitHub;
- installs the JDK, Maven, Postgres, Python, hostapd/dnsmasq/iptables;
- sets up SSH, passwordless sudo for `pi`, and serial port access (`dialout` group);
- builds PaulaUploader into `~/paulauploader`;
- installs Tomcat 8.5.100 in `~/pauladeployer/tomcat`, then clones and builds PaulaDeployer into it, and installs the `pauladeployer-tomcat` service;
- sets up the field hotspot on `wlan0` and the client on `wlan1`;
- adds the port redirects 80→8080 and 443→8443 to `/etc/rc.local`, then reboots.

Options are set as variables in front of the command (all documented at the top of `provision-pi.sh`):

```
FIELD_PASSWORD='hotspot-password'  ./provision-pi.sh      # WPA2 hotspot instead of an
open one
FACTORY_WIFI_SSID='OfficeWifi' FACTORY_WIFI_PASSWORD='...' ./provision-pi.sh
```

For a field Paula, give the phone hotspot and the Factory tunnel address in the same run (section 8 explains both):

```
FIELD_PASSWORD='...' FACTORY_WIFI_PASSWORD='...' \
PHONE_HOTSPOT_SSID='Ari's iPhone' PHONE_HOTSPOT_PASSWORD='...' \
FACTORY_TUNNEL_IP=10.98.0.4 \
./provision-pi.sh
```

The office WiFi joined in 3.2 is used for `wlan1` automatically, but its password is never detected. For a secured network, give `FACTORY_WIFI_PASSWORD`.

At the end the script says HTTPS is not set up yet; that is section 7. The Pi reboots, which takes 30–45 s.

5. PI Trust the Pi on the NUC (once per Pi)

On a new Pi the first run always skips the esptool/bootloader download, because the Pi's SSH key is not trusted on the NUC yet. After the reboot:

```
ssh-copy-id -i ~/.ssh/chilhuacle.pub ari@192.168.1.138    # NUC password once
cd ~/PaulaUploader && ./provision-pi.sh                    # second run: only the esptool
fetch is new
```

Every step of the script is safe to repeat.

6. COMPUTER Flash Paula's ESP32 (Wally) with Paula.ino

Plug Wally into the computer by USB and run:

```
cd ~/Data/DigitalStables/Projects/Paula
arduino-cli compile --fqbn esp32:esp32:esp32 \
  --libraries ~/Data/DigitalStables/ProjectsSupport/Esp32ArduinoLib/libraries .
arduino-cli upload  --fqbn esp32:esp32:esp32 --port /dev/ttyUSB0 .
arduino-cli monitor --port /dev/ttyUSB0 --config baudrate=115200   # optional: watch it boot
```

Notes:

- Board `esp32:esp32:esp32`, 4 MB flash; the sketch uses about 91 %.
- LoRa runs at 433 MHz, SF9, 125 kHz, with CRC on. Every device on the farm must match these settings.
- The Arduino IDE works too, with the same board and libraries.

Then plug Wally into the **Pi**. To check that the Pi sees it, run on the Pi:

```
java -jar ~/paulauploader/target/paulauploader.jar diagnose-ports    # should name Wally's port
```

> Paula identifies packets by their size only. If any LoRa struct changes size, check that no two structs end up the same size (see the Paula project's `CLAUDE.md`).

7. HTTPS: certificate and phones

Why: phones only give their GPS to `https` pages. An iPhone has no way around this. The Nearby Devices map uses the phone's GPS as Paula's position, so without HTTPS the map works, but you have to tap the map to show where you are.

How: Digital Stables runs its own small certificate authority.

- One **root certificate** lives on Ari's computer. Each phone installs and trusts it once.
- Each Paula gets a certificate signed by that root, for `<hostname>.local`, `<hostname>` and `192.168.50.1`.

After that, `https://paula4.local/` opens with no warnings. It needs no internet, no domain and no renewal for 825 days (the longest an iPhone accepts).

The scripts are in the PaulaUploader repository, folder `https/`:

Script	Where	What it does
`make-paula-cert.sh`	Computer	Creates the root certificate on its first run, signs a certificate for one Paula and packs it as `paula-https.p12` for Tomcat. With `--install` it also copies the file to the Pi and runs the next script there.
`enable-https.sh`	Pi	Adds an HTTPS connector on port 8443 to Tomcat's `server.xml` (only once; it backs up the original first). Redirects 443→8443, now and at every boot, then restarts Tomcat and checks that HTTPS answers. Plain http keeps working.

COMPUTER 7.1 Make and install the Paula's certificate

The Pi must be on the same network as the computer, and its PaulaUploader checkout must be up to date: `cd ~/paulauploader && git pull` on the Pi.

```
cd ~/Data/DigitalStables/PaulaUploader/https
./make-paula-cert.sh paula4 --install
```

- The first time ever, this also creates `~/DigitalStablesCA/rootCA.key` and `rootCA.crt`.
- To reach the Pi by IP instead of `paula4.local`: `PI_ADDRESS=192.168.1.50 ./make-paula-cert.sh paula4 --install`
- To add another name or IP to the certificate, list it after the hostname: `./make-paula-cert.sh paula4 172.20.10.2 --install`
- Without `--install`, the script prints the two manual commands (an `scp` and an `ssh`).

Success ends with: `HTTPS is up: https://paula4.local/ and https://192.168.50.1/`.

> **Back up `~/DigitalStablesCA/rootCA.key`** somewhere safe and private. Anyone with it can make certificates that every Digital Stables phone trusts. If it is lost, a new root has to be made and installed again on every phone, and every Paula needs a new certificate.

PHONE 7.2 Install the root certificate on a phone (once per phone)

Send `~/DigitalStablesCA/rootCA.crt` to the phone, by AirDrop, email or a download link.

iPhone

1. Open the file. iOS says "Profile Downloaded".
2. *Settings → General → VPN & Device Management → Digital Stables Paula Root CA →* **Install**.
3. *Settings → General → About → Certificate Trust Settings* → switch on **full trust** for *Digital Stables Paula Root CA*. This step is easy to forget, and without it the certificate does nothing.

Android

- *Settings → Security → Encryption & credentials → Install a certificate → CA certificate*, then choose `rootCA.crt`. The menu names vary by brand; searching Settings for "certificate" finds it.
- Chrome on Android uses it straight away.

Computer (optional, for testing)

- Linux Chrome: *Settings → Privacy and security → Security → Manage certificates → Authorities → Import*, tick "Trust this certificate for identifying websites".

PHONE 7.3 Check

Open `https://paula4.local/`. It should load with a normal padlock and no warning. Then press **Nearby Devices**: the phone asks to share its location; allow it.

7.4 Renewing and changing

- The certificate lasts 825 days, and `enable-https.sh` prints the expiry date. To renew, re-run `./make-paula-cert.sh paula4 --install`; phones need nothing new.

- If the Pi's hostname changes, make a new certificate for the new name.
- The root certificate is valid for 10 years. The script never remakes it while `rootCA.key` exists.
- If `provision-pi.sh` is re-run on a Pi that already has `paula-https.p12`, it turns HTTPS on again by itself.

8. Field networking: phone hotspot and Factory tunnel

In the field, `wlan1` joins the phone's 4G hotspot. This gives Paula internet (NTP time, Stream 2 Chat through the Factory tunnel), and the phone reaches PaulaDeployer on the same hotspot. Map imagery is downloaded by the phone itself.

PI 8.1 The phone's hotspot

`provision-pi.sh` adds the phone's hotspot to `wlan1` when given `PHONE_HOTSPOT_SSID` and `PHONE_HOTSPOT_PASSWORD`:

- the home/factory WiFi gets `priority=10` and the hotspot `priority=1`, so Paula uses the home WiFi whenever it can see it, and the phone otherwise;
- the hotspot name is written in hex. An iPhone's name contains a curly apostrophe (`Ari’s iPhone`) that has to match byte for byte, so copy it from the phone (*Settings* → *General* → *About* → *Name*) rather than typing it;
- on the iPhone, turn on *Personal Hotspot* → *Maximize Compatibility*, so the hotspot uses 2.4 GHz, which every Pi can see.

To add or change it later by hand: `sudo nano /etc/wpa_supplicant/wpa_supplicant.conf` and add a `network={ ... priority=1 }` block. To get the exact name, scan from the Pi: `sudo iw dev wlan1 scan | grep SSID`.

PaulaDeployer's **(i)** popup shows which network `wlan1` is on, and has buttons to force one network or go back to Automatic. That helps at the edge of the home WiFi's range.

8.2 Factory tunnel (for Stream 2 Chat from the paddock)

Away from home, Paula reaches the FactoryChat broker (FACTORY2, 192.168.1.138:1883) through WireGuard:

```
iPhone hotspot ── Paula ═ WireGuard ═ chilhuacle relay ═ OpenWrt One (wg_remote) ── FACTORY2:1883
```

Each Paula needs its own **tunnel address**, `10.98.0.N`. Taken so far: .1 the router, .2 Ari's phone, .3 Paula3. Use .4 for the next Paula and keep a list.

PI 8.2.1 On the Pi: done by provision-pi.sh

With `FACTORY_TUNNEL_IP=10.98.0.N`, the script:

- installs `wireguard-tools`;
- makes the Paula's WireGuard key on the Pi. A re-run keeps the existing key, because the router knows the Paula by it;
- writes `/etc/wireguard/factory.conf`. Only FACTORY2 and the router's tunnel address go through the tunnel; everything else (map tiles, NTP) uses the hotspot directly;

- installs `/usr/local/sbin/factory-tunnel` with `factory-tunnel.timer`, which runs it every 15 s:
  - on the home WiFi (`MainRouter24`/`MainRouter`, or `FACTORY_HOME_SSIDS`) it keeps the tunnel **down**, so chat goes straight over the LAN and works even with the satellite link down;
  - on any other network it brings the tunnel **up**;
  - after a network change it renews the DHCP address;
- leaves the public key and address in `~/factory-tunnel.pub` for the router step.

Its log: `journalctl -t factory-tunnel`.

COMPUTER 8.2.2 On the router: add-paula-to-router.sh (once per Paula)

The router has to know the new Paula. From the computer, on the home network:

```
cd ~/Data/DigitalStables/PaulaUploader/tunnel
./add-paula-to-router.sh paula4          # or: PAULA_ADDRESS=192.168.1.171 ./add-paula-to-router.sh paula4
```

It reads `~/factory-tunnel.pub` from the Pi, then on the OpenWrt One (`root@192.168.1.1`, key `~/.ssh/chilhuacle`):

- **adds the peer** to `wg_remote`. It is saved with `uci` and added live with `wg set`, so there is no network restart and the house WiFi stays up;
- **adds a firewall door**, `Cleotilde-Door-FactoryChat-Paula4`, that lets this Paula's address reach only 192.168.1.138 tcp 1883, then reloads the firewall;
- refuses if the address already belongs to another peer, and prints the commands to undo it.

The chilhuacle relay needs nothing per Paula. Optionally, add the address to `~/Teleonome/TunnelPeers.json` on the Teleonome Pi so the Network Rules panel shows the name.

Test in the field: hotspot on, and Paula joins it within about 15 s. The tunnel comes up within another 15 s (`journalctl -t factory-tunnel` says "tunnel up"), the router's `wg show wg_remote` shows a recent handshake, and Stream 2 Chat works. Full design: doc 07, `/home/ari/Data/OpenWrtOne/documents/07-network-rules-design.md`.

8.3 How the phone reaches PaulaDeployer

Phone is on	Open	Notes
Paula's own hotspot (SSID = hostname)	`https://192.168.50.1/`	Always works. The phone has no internet this way, so the map has no imagery: devices and lines still show, on a grey background.
The phone's own 4G hotspot (Paula joined it)	`https://paula4.local/`	Map imagery comes over 4G. If `.local` doesn't resolve, use the `wlan1` address shown in the (i) popup with `http://`; the certificate has no hotspot IP unless it was added in 7.1.
Home/office WiFi	`https://paula4.local/`	For bench work.

9. Paula's secret and clock

Paula's commands to Daffodils (WiFi on/off) carry a TOTP code made from the **customer's secret**, and Paula only accepts weather forecasts carrying a valid code. A Daffodil rejects any command whose code is wrong, or whose time is more than 15 minutes off its own clock.

PHONE In PaulaDeployer → **Nearby Devices**, below the list:

1. Plug one of the customer's Daffodils into the Pi (jumper on **USB**).
2. Press **Check**, then **Copy secret to Paula** if it says different. The secret itself is never shown.
3. **Paula's clock** is shown above it. The Pi sets Paula's clock each time it opens the port, but only when the Pi's own clock is internet-synced. If the Pi isn't synced, press **Set Pi Time** in the header, then **Set Paula's clock**.

> When the secret and clock are right, Paula's LEDs 2–3 turn green within a few minutes, once Annabelle's next weather forecast is received with a valid code.

10. Final checks

Check	Expected
Join the hotspot (SSID = hostname), then `ssh pi@192.168.50.1`	Logs in
`java -jar ~/paulauploader/target/paulauploader.jar diagnose-ports`	No Java error; Wally's port named
`ls ~/.arduino15/packages/esp32/tools/esptool_py/3.0.0/esptool.py`	Exists (section 5 worked)
`https://paula4.local/` on a phone with the root certificate	Padlock, no warning, PaulaDeployer loads
`http://192.168.50.1/`	Still loads (plain http keeps working)
Nearby Devices	Map with the Paula logo at your position; devices start appearing as they send
Nearby Devices → Check secret	"Paula has the same secret"
Factory webapp → Products → Send Deploy Package	The new Paula is listed as reachable within a few minutes (it registers itself)
On the phone's hotspot: `journalctl -t factory-tunnel` on Paula	"tunnel up" within ~30 s; Stream 2 Chat reaches a chat

(i) popup	wlan0/wlan1 addresses, Wi-Fi network, tunnel state, version 0.9.1 or later

11. Using the Nearby Devices map

The question it answers: **can a device placed here get its packets to Annabelle, the gateway?** Open it from the **Nearby Devices** button.

- **Annabelle banner (top).** How strongly Paula hears Annabelle's weather forecast at this spot:
  - green: strong, a device here should reach Annabelle;
  - amber: fair, little margin;
  - red: weak, the device may need a relay;
  - grey: not heard yet. Forecasts go out every few minutes; nothing after about 10 minutes means Annabelle doesn't reach this spot.

  This is a strong hint, not proof, because Annabelle may transmit more strongly than a Daffodil. The planned "ping" (phase 2) will confirm both directions.
- **Map.** The Paula logo marks your phone's position. Each device sits where its data packets say it is: Daffodil, Langley, Chinampa and Gloria all broadcast their coordinates.
  - A line runs from where you were when Paula heard the device to the device, coloured by signal strength.
  - Lines fade after 3 minutes and become dashed after 15 minutes, since you have probably moved since.
  - The view follows new devices until you drag the map; **Show all** brings it back.
- **List.** Signal bars, RSSI/SNR, time since last heard, packet count and distance. Daffodils keep their **WiFi on/off** button, which keeps the device awake with WiFi for 15 minutes, as before.
- **Save here.** Standing next to a device, this sets its map position to yours. Use it for Annabelle (its forecast carries no position) and for any device with missing or wrong coordinates. A saved position overrides the broadcast one until you press **Clear**. Saved positions are kept on the Pi in `/home/pi/paulauploader/signalmap-positions.json`.
- **Capture file.** Every session is saved under `captures/lorasniffer/` (linked at the bottom of the screen). Each packet line includes the device's position and yours, for example `at -37.13,144.47 paula -37.13,144.47 ~5m`, so a walk can be read back later.

Signal cut-offs (SF9, sensitivity about −126 dBm): green ≥ −95 dBm, amber −95 to −110, red below −110. Signal bars: 4 at ≥ −85, 3 at ≥ −95, 2 at ≥ −110, otherwise 1.

11.1 Ping from here

The banner is a hint; the ping is the test. The blue **Ping from here** button makes Paula send one ping to every device. Every device that is listening answers within about 5 seconds and reports:

- **how strongly it heard the ping**, which is how a device placed where you stand would be heard;

- for Daffodils, **how well it hears Annabelle** (from its last weather forecast), so it can be judged as a relay;
- its own position, so it lands on the map even without "Save here".

The ping goes out at **5 dBm** by default: the power of a Daffodil, Langley or Gloria. Paula and Annabelle normally use 17 dBm, about 12 dB stronger, which would make every spot look better than it is. Choose 17 dBm in the selector only to test for a 17 dBm device (Chinampa).

The verdict panel says one of:

Verdict	Meaning
✓ A device here reaches Annabelle (green / amber / red by margin)	Annabelle answered. The RSSI it heard the ping at is the uplink a device here would have.
⇄ Annabelle did not answer - a relay is needed here (purple)	Other devices heard the ping and also hear Annabelle. The best relay is named: the one whose weaker link (ping or Annabelle) is strongest.
✗ Annabelle did not answer / Nobody answered (grey)	No listening device heard this spot.

On the map, each answer gets a thick line from where you pinged, coloured by how the device heard you. A relay's own link to Annabelle is drawn dotted. Each ping leaves a dot at its spot in the verdict's colour, so walking around and pinging builds a coverage map. Every ping and answer is also written to the capture file (`PING #n` and `answers PING #n` lines).

Who answers: Annabelle, and Daffodils and Langleys that are awake. A sleeping Daffodil or Langley only listens for a few seconds after its VitalSigns, so it almost never answers. Chinampa and Gloria don't answer yet, since their firmware doesn't have the ping. Firmware older than this ignores pings safely.

Firmware needed: `Paula.ino` (Version 1 or later), `Annabelle.ino` (Version 12 or later), and `Daffodil.ino` and `Langley.ino` from 3 November 2026 or later, built with the library files `LinkAckRecord.h` and `DeviceCommandRecord.h` (which has the new `DEVICE_COMMAND_PING`). Pings carry a code from the customer's secret and Paula's time, like the WiFi commands, so Paula's secret and clock must be right (section 9).

Sleeping Daffodils only show up when they wake and send. Devices Paula heard before the screen opened are listed from Paula's own table, marked "before this screen", without a line.

12. Updating a Paula later

What	How
PaulaDeployer	**COMPUTER** commit and push. **PI** `cd ~/pauladeployer-src && git pull && mvn package -Dmaven.antrun.skip=true && cp target/ROOT.war ~/pauladeployer/tomcat/webapps/ROOT.war && sudo systemctl restart pauladeployer-tomcat`. Always restart the service rather than relying on a hot redeploy (jSerialComm can only load once per process). New files must be `git add`-ed: a missing `leaflet.css` breaks the map.
PaulaUploader	**PI** `cd ~/paulauploader && git pull && mvn package`

Paula.ino (Wally)	**COMPUTER** as in section 6
HTTPS certificate	**COMPUTER** `./make-paula-cert.sh <hostname> --install` (section 7.4)

After a PaulaDeployer update the page loads fresh by itself: the `?v=` tags on its CSS and JS change with each version.

13. Troubleshooting

Map and HTTPS

Symptom	Cause	Fix
Map tiles scattered or overlapping all over the screen	`assets/leaflet/leaflet.css` not deployed (it was left out of a commit once)	`git add src/main/webapp/assets/leaflet`, push, update the Pi, hard-refresh
"Phone location blocked - this page is http"	Page opened with `http://`	Use the `https://` link in the note; install the root certificate if not done (7.2)
Certificate warning on `https://`	Root not installed or not trusted on this phone; or the address used isn't in the certificate	iPhone: Certificate Trust Settings switch (7.2). Use `<hostname>.local` or `192.168.50.1`, or add the address (7.1)
"Phone location blocked - allow location" on `https://`	Location permission refused earlier	iPhone: *Settings → Privacy → Location Services → Safari/Chrome → While Using*, and allow the site
`https://` doesn't answer at all	No connector or no 443 redirect (e.g. rc.local not run)	On the Pi: `bash ~/paulauploader/https/enable-https.sh`; it reports the problem. Tomcat log: `~/pauladeployer/tomcat/logs/catalina.out`
Tomcat won't start after enabling HTTPS	`server.xml` damaged	Restore `conf/server.xml.before-https` and re-run `enable-https.sh`
Map grey, no imagery	The phone has no internet (e.g. on Paula's own hotspot)	Use the phone's 4G hotspot (8.3). Devices and lines still work
Device in the list but not on the map	It broadcasts no position (0,0, or a type without one, e.g. Seedling)	Stand next to it and press **Save here**; or set its GPS on the device's own web page

Annabelle never appears	Out of range, or Paula's secret is wrong (forecasts with a bad code are ignored)	Check the secret (section 9); compare with the Lora Sniffer, which shows "BAD CODE"
Stream 2 Chat fails away from home	Tunnel not up, or the router doesn't know this Paula	`journalctl -t factory-tunnel` on Paula; `wg show wg_remote` on the router. Run `add-paula-to-router.sh` if the Paula's key isn't listed (8.2.2)
Paula never joins the phone's hotspot	Name mismatch (curly apostrophe), or the hotspot is on 5 GHz	Copy the name from the phone; turn on Maximize Compatibility (8.1)
Ping: nobody answers, even next to Annabelle	Old firmware on Annabelle/Paula, wrong secret, or Paula's clock more than 15 min off (pings are rejected like commands)	Flash the 3 Nov 2026 firmware; section 9 for secret and clock
Ping button: "Paula did not answer"	Paula.ino older than the ping	Flash Paula.ino (section 6)
WiFi on/off never completes	Paula's clock more than 15 minutes off, wrong secret, or the device is asleep	Section 9; a sleeping device answers when it next wakes

Provisioning (from the PaulaUploader README)

Symptom	Cause	Fix
Field hotspot comes up, then drops seconds later	A failing command in `rc.local` made the whole script exit; systemd killed its processes	Every `rc.local` line is guarded with `|| true`; `iptables` must be installed
USB WiFi never comes up, ifup/ifdown hang	`allow-hotplug` on the dongle races `rc.local`	No `allow-hotplug` for the dongle
`Operation not possible due to RF-kill`	NetworkManager disabled, so nothing clears rfkill	`rfkill unblock all` (in `rc.local`)
wlan0 and wlan1 swap after reboots	Kernel naming order not fixed	The udev rule pins the built-in radio to `wlan0`
Provisioning hangs 30+ minutes, SSH dies	Network work after NetworkManager was disabled	All downloads run before NetworkManager is touched; run from the Pi's own keyboard
esptool/bootloader fetch skipped	Pi's key not trusted on the NUC	Section 5
Paula "not reachable" in the factory dropdown	Registered without `.local`	Fixed in current code (appends `.local`)
Flash "succeeds" but the device wasn't flashed	`upload.sh` without `set -e`	Fixed: every generated `upload.sh` has `set -e`

14. Reference

Item	Value
Paula hotspot address	`192.168.50.1` (DHCP 192.168.50.10–100)
PaulaDeployer	`http://` :80→8080, `https://` :443→8443; service `pauladeployer-tomcat`
Tomcat	`~/pauladeployer/tomcat` (8.5.100); certificate `conf/paula-https.p12`
PaulaDeployer source on the Pi	`~/pauladeployer-src`
PaulaUploader on the Pi	`~/paulauploader` (also holds `https/` and `docs/`)
Field captures	`/home/pi/paulauploader/captures/<kind>/`, browsable at `/captures/`
Saved map positions	`/home/pi/paulauploader/signalmap-positions.json`
Certificate authority	Ari's computer, `~/DigitalStablesCA/`: `rootCA.key` (private), `rootCA.crt` (for phones), `paulas/<hostname>/`
Factory tunnel	Paula: `/etc/wireguard/factory.conf`, `factory-tunnel.timer`, `~/factory-tunnel.pub`. Addresses 10.98.0.N (.1 router, .2 Ari's phone, .3 Paula3). Endpoint `104.154.239.253:51821`
Wally USB ID	CP2104, 10C4:EA60
LoRa	433 MHz, SF9, BW 125 kHz, CRC on
Office NUC	`192.168.1.138` (user `ari`)